When Search Is Not Enough: Diagnosing Failure Before Constructing Verified Reasoning Capabilities

Chimdi Walter Ndubuisi

University of Missouri · ARC Prize Research Summit, 23 October 2026 · poster abstract

Many reasoning systems respond to failure by generating more programs, refining candidates or adding test-time computation; these strategies are bounded by the representations and operations available to search. We study a complementary question: can a reasoner tell whether a failure reflects insufficient search, an excluded representation, a missing executable capability or insufficient evidence, and adapt only that component without compromising the evidence used to judge it?

We present CORA, a framework for failure-conditioned construction and verification of task-local executable extensions. A candidate is built from auditable failure traces, installed provisionally into the unchanged native reasoner, and checked by demonstration replay, adaptive leave-one-out rebuilt per fold, independent target evaluation and removal controls; abstention is a legitimate outcome.

A frozen synthetic experiment produced 27 of 30 complete end-to-end witnesses (12 of 30 for the earlier engine) within its declared grammar. On 60 real ARC-AGI-2 training-development tasks, four automatically constructed growth relations compiled and installed (none before), and one was accepted and used by the native engine. None survived adaptive leave-one-out (0 of 14 folds), so no new ARC solve is claimed. An instrumented failure funnel on the same 60 tasks, with held-out answers read only by an independent evaluator, then located where the native reasoner fails. Its leave-one-out verifier certified 6 answers, all correct; switching it off would add 2 correct and 26 wrong answers. In 27 tasks every program that fit the demonstrations was wrong: mostly memorized constant growth patterns, rebuilt differently by each leave-one-out fold. Neither the reasoner's own relational re-search nor three times the search budget repaired any of them, and the view-admission failure we had first traced by hand explains only 7 of 76 failed folds. The obvious view repairs were tested on synthetic controls first: forcing a view to be searched first gained no task and harmed five of 24, and a fold-local alternative gained one and harmed four of 41, so neither was adopted. Frozen neural guides chose views no better than the fixed order (Qwen2.5-1.5B 19 of 35, I-JEPA 3 of 35, pre-registered).

We then implemented the construction step (CORA-FHR): a declared grammar of typed growth operators, the reasoner's own memorized-growth record as evidence, reversible installation into the unchanged engine, and leave-one-out that rebuilds the operator in every fold. On 27 synthetic development tasks, one constructed operator, absent from the native and existing constructor vocabularies, was rebuilt in every fold and gave a correct certified prediction. There were no wrong certificates, and matched extra search gained nothing. A failure-blind constructor reached the same result, so the contribution of the failure record itself is not yet established; the prospective test is sealed.

We outline a domain-portable control protocol, whose general failure-type controller is not yet implemented, and a prospective biological-perturbation test (untested). Neither biological transfer nor real ARC invention has been demonstrated.

Keywords: ARC-AGI-2; program synthesis; failure diagnosis; capability construction; verification; leave-one-out.

Evidence tiers: synthetic (frozen, run once); real development (training demonstrations only, no test output read); forensic (subsets examined after the failure was seen). The certified training score is unchanged at 185/1,000, and the native solver's official Kaggle ARC-AGI-2 public score is 0.00 (October 2026). A separately labelled neural baseline (a reproduction of a public NVARC test-time-training pipeline, not CORA) was submitted on 14 October; its score is pending and is not a CORA result. A 120-task prospective pilot is frozen and has not been run. Every number is indexed with its commit and hash in `records/CORA_SUMMIT_POSTER_EVIDENCE_20261012.md` and `records/CORA_FAILURE_FUNNEL_20261014.md`.